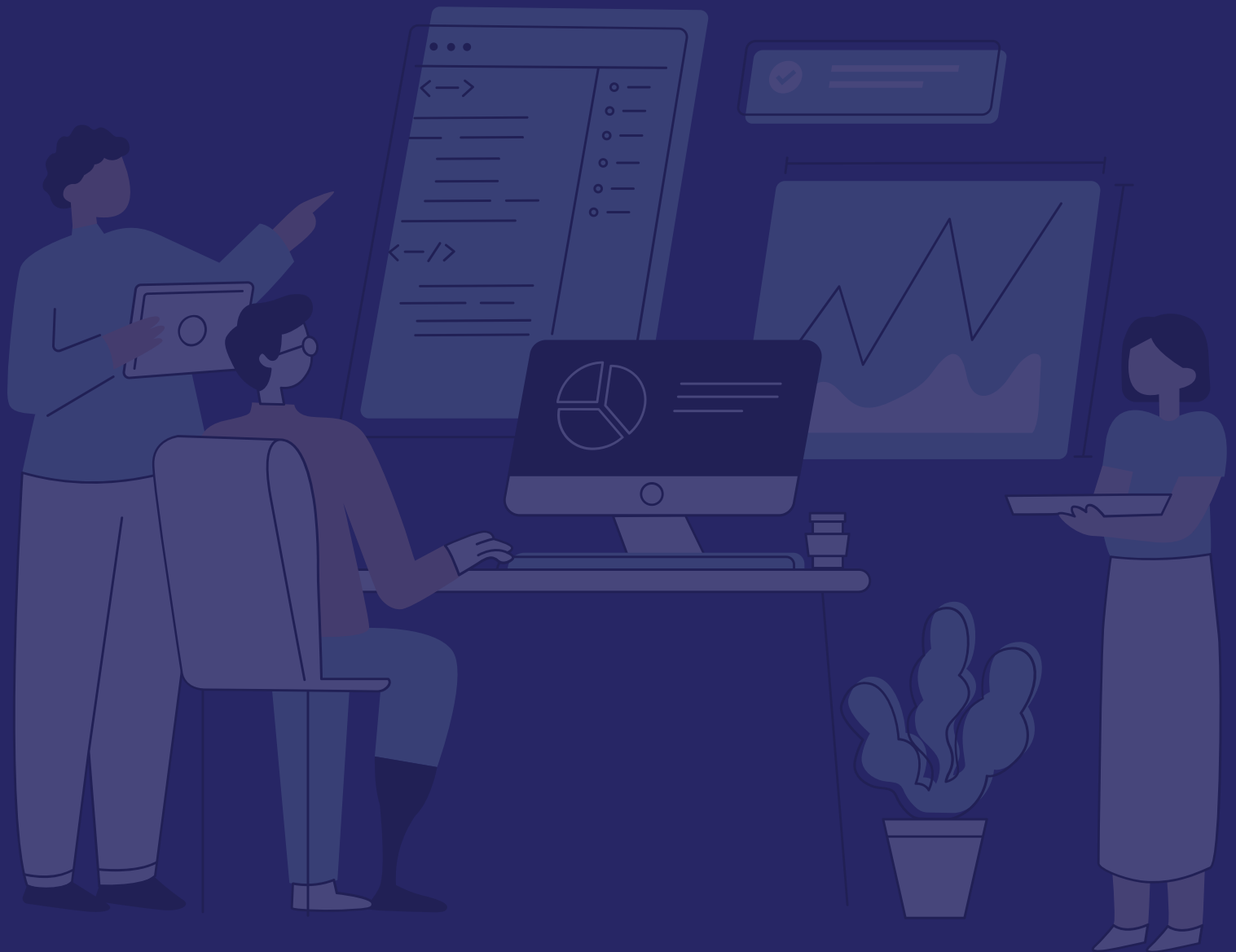




Saathi

A Companion app for Senior Citizens



SE [CS3001]

Team-19

Milestone - 5

Software Engineering Project

Team Profile

S.No	Name	Student Email
1.	Devaanshi Chaudhary	21f3001136@ds.study.iitm.ac.in
2.	Advait Vats	21f1005152@ds.study.iitm.ac.in
3.	Somsubhra De	22f3002680@ds.study.iitm.ac.in
4.	Drishti	21f1004302@ds.study.iitm.ac.in
5.	Shourjya Bhattacharjee	21f1000757@ds.study.iitm.ac.in
6.	Karthik Ganesh	21f2000775@ds.study.iitm.ac.in
7.	Jatin Yadav	22f2001018@ds.study.iitm.ac.in

Overview of Milestone 5: Testing & Integration

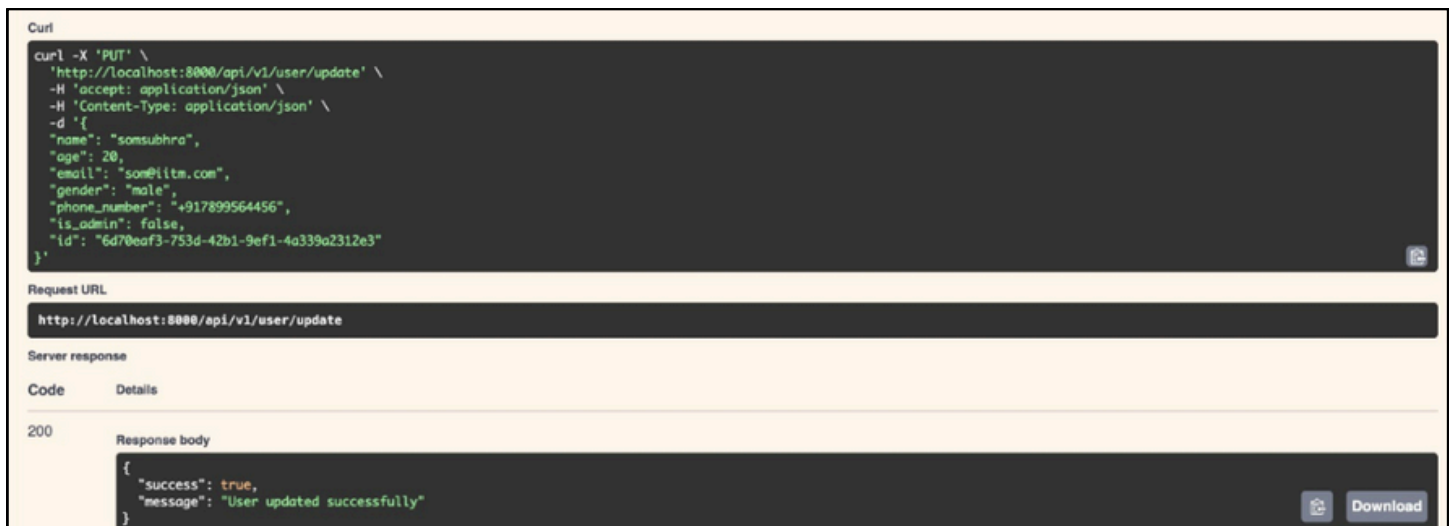
In this milestone, we focus on validating the **functionality, reliability and performance of the APIs** that drive the Digital Solution for Senior Citizens platform. The goal of this project is to provide senior citizens with an accessible, user-friendly digital experience to overcome various challenges they face in daily life. To ensure the platform operates smoothly and meets its intended goals, extensive testing of its APIs is necessary.

We have carefully designed test cases for 7 key APIs, each designed to handle different aspects of the platform's functionality. The test cases are structured to validate **API responses**, ensuring that the inputs, expected outputs and actual outputs are consistent, ensure the **other functionalities**, for eg. **integration with GenAI** functions as intended. Furthermore, **unit tests** were written using **pytest** to automate testing and improve the overall development process. In the following sections, we will outline the test cases for the APIs, provide detailed inputs and expected outputs. No issues were discovered.

Test Cases

1. Update User Details

```
def test_update_user():
    response = client.post("api/v1/user/update", json={
        "name": "somsubhra",
        "age": 20,
        "email": "som@iitm.com",
        "gender": "male",
        "phone_number": "123",
        "id": "6d70eaf3-753d-42b1-9ef1-4a339a2312e3",
    })
    print(response.json())
    assert response.status_code == 200
    assert response.json()["message"] == "User updated successfully"
```



Page being tested: <http://localhost:8000/api/v1/user/update>

Inputs:

- Request Method: **PUT**
- JSON Body

```
{
  "name": "somsubhra",
  "age": 20,
  "email": "som@iitm.com",
  "gender": "male",
  "phone_number": "+917899564456",
  "is_admin": false,
  "id": "6d70eaf3-753d-42b1-9ef1-4a339a2312e3"
}
```

- **Headers**

Content-Type: application/json

accept: application/json

Expected Output:

- **HTTP Status Code: 200**

- **JSON Response:**

```
{  
  "success": true,  
  "message": "User updated successfully"  
}
```

Actual Output:

- **HTTP Status Code: 200**

- **JSON Response:**

```
{  
  "success": true,  
  "message": "User updated successfully"  
}
```

Result: Success

2. Create Reminder

```
def test_create_reminder():  
    response = client.post("api/v1/reminder/create", json={  
        "title": "Morning Medicine",  
        "frequency": "Daily",  
        "time_of_day": "04:30:00.000Z",  
        "is_active": True,  
        "user_id": "a7e1190e-b529-4e55-996e-fc8d700f6c1e"  
    })  
    print(response.json())  
    assert response.status_code == 200  
    assert response.json()["message"] == "Reminder created successfully"
```



Page being tested: <http://localhost:8000/api/v1/reminder/create>

Inputs:

- Request Method: **POST**
- **JSON Body**

```
{
  "title": "Morning Medicine",
  "frequency": "Daily",
  "time_of_day": "04:30:00.000Z",
  "is_active": true,
  "user_id": "6d70eaf3-753d-42b1-9ef1-4a339a2312e3"
}
```

- **Headers**

Content-Type: application/json

accept: application/json

Expected Output:

- **HTTP Status Code:** 200
- **JSON Response:**

```
{
  "success": true,
  "message": "Reminder created successfully"
}
```

Actual Output:

- **HTTP Status Code:** 200
- **JSON Response:**

```
{
  "success": true,
  "message": "Reminder created successfully"
}
```

Result: Success

3. Update Reminder

```
def test_update_reminder():
    response = client.post("api/v1/reminder/update", json={
        "id": "a7e1190e-b529-4e55-996e-fc8d700f6c1e",
        "title": "Morning Medicine",
        "frequency": "Daily",
        "time_of_day": "05:30:00.000Z",
        "is_active": True,
    })

    print(response.json())
    assert response.status_code == 200
    assert response.json()["message"] == "Reminder updated successfully"
```



Curl

```
curl -X 'PUT' \
  'http://localhost:8000/api/v1/reminder/update' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Morning Medicine",
    "frequency": "Daily",
    "time_of_day": "05:30:00.000Z",
    "is_active": true,
    "id": "a7e1190e-b529-4e55-996e-fc8d700f6c1e"
  }'
```

Request URL

http://localhost:8000/api/v1/reminder/update

Server response

Code	Details
200	Response body <pre>{ "success": true, "message": "Reminder updated successfully" }</pre>

Download

Page being tested: <http://localhost:8000/api/v1/reminder/update>

Inputs:

- Request Method: **POST**
- JSON Body

```
{
  "title": "Morning Medicine",
  "frequency": "Daily",
  "time_of_day": "05:30:00.000Z",
  "is_active": true,
  "id": "a7e1190e-b529-4f55-996e-fc8d708fc61e"
}
```

- Headers

Content-Type: application/json

accept: application/json

Expected Output:

- HTTP Status Code: 200

- JSON Response:

```
{
  "success": true,
  "message": "Reminder updated successfully"
}
```

Actual Output:

- HTTP Status Code: 200
- JSON Response:

```
{
  "success": true,
  "message": "Reminder updated successfully"
}
```

Result: Success

4. Create User

```
def test_create_user():
    response = client.post("api/v1/user/create", json={
        "name": "som",
        "age": 20,
        "email": "som@iitm.com",
        "gender": "male",
        "phone_number": "+917899564456",
        "is_admin": False
    })

    print(response.json())
    assert response.status_code == 200
    assert response.json()["message"] == "User created successfully"
```



Curl

```
curl -X 'POST' \
  'http://localhost:8000/api/v1/user/create' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "som",
    "age": 20,
    "email": "som@iitm.com",
    "gender": "male",
    "phone_number": "+917899564456",
    "is_admin": false
  }'
```

Request URL

```
http://localhost:8000/api/v1/user/create
```

Server response

Code	Details
200	Response body <pre>{ "success": true, "message": "User created successfully" }</pre>

Page being tested: <http://localhost:8000/api/v1/user/create>

Inputs:

- Request Method: **POST**
- JSON Body

```
{
  "name": "som",
  "age": 20,
```

```
"email": "som@iitm.com",  
"gender": "male",  
"phone_number": "+917899564456",  
"is_admin": false  
}
```

- **Headers**

Content-Type: application/json

accept: application/json

Expected Output:

- **HTTP Status Code:** 200

- **JSON Response:**

```
{  
    "success": true,  
    "message": "User created successfully"  
}
```

Actual Output:

- **HTTP Status Code:** 200

- **JSON Response:**

```
{  
    "success": true,  
    "message": "User created successfully"  
}
```

Result: Success

5. Create Message

```
def test_create_message():  
    response = client.post("api/v1/message/create", json={  
        "sender_id": "cd8989cc-3aa2-40f1-8739-7ede942874d1",  
        "recipient_id": "28397812-9562-4027-8ef8-986c86c16ba5",  
        "content": "Hey",  
    })  
    print(response.json())  
    assert response.status_code == 200  
    assert response.json()["message"] == "Message created successfully"
```



Page being tested: <http://localhost:8000/api/v1/message/create>

Inputs:

- Request Method: **POST**
- JSON Body

```
{
  "sender_id": "cd89809c-3ca2-40f1-8739-7ede942874d1",
  "recipient_id": "28379812-9562-4027-8ef8-986c8c61b6d5",
  "content": "Hey"
}
```

- Headers

Content-Type: application/json

accept: application/json

Expected Output:

- HTTP Status Code: 200
- JSON Response:

```
{
  "success": true,
  "message": "Message created successfully"
}
```

Actual Output:

- HTTP Status Code: 200
- JSON Response:

```
{
  "success": true,
  "message": "Message created successfully"
}
```

Result: Success

6. Transcribe Audio

```
def test_transcribe_audio_success():
    """
    Tests the happy path for the /transcribe-audio endpoint.
    It mocks the external Groq API call to ensure the test is fast and isolated.
    """
    with patch('main.groq_client') as mock_groq:
        # Arrange: Configure the mock to return a successful transcription
        mock_transcription_result = MagicMock()
        mock_transcription_result.text = "This is a test transcription."
        mock_groq.audio.transcriptions.create.return_value = mock_transcription_result

        with open(TEST_AUDIO_PATH, "rb") as audio_file:
            files_to_upload = {'audio_file': (f'test_audio.mp3', audio_file, 'audio/mpeg')}
            response = client.post("/transcribe-audio", files=files_to_upload)

            # Expected Output
            expected_status_code = 200
            expected_json = {
                "transcribed_text": "This is a test transcription."
            }
            # Actual Output
            actual_status_code = response.status_code
            actual_json = response.json()

            # Result
            assert actual_status_code == expected_status_code
            assert actual_json == expected_json

            # Verify that the mocked function was called once
            mock_groq.audio.transcriptions.create.assert_called_once()

    print("\n[ API: /transcribe-audio (Success) ]")
    print(f"[ Inputs: Valid MP3 file ('{TEST_AUDIO_PATH.name}') ]")
    print(f"[ Expected Output: Status {expected_status_code}, Body {expected_json} ]")
    print(f"[ Actual Output: Status {actual_status_code}, Body {actual_json} ]")
    print("[ Result: Success ]")
```

```
===== test session starts =====
platform win32 -- Python 3.11.5, pytest-8.3.3, pluggy-1.5.0 -- d:\conda\python.exe
cachedir: .pytest_cache
rootdir: D:\IITMSE
plugins: anyio-4.3.0, langsmith-0.4.8, asyncio-1.1.0, mock-3.14.1, tornasync-0.6.0.post2, trio-0.8.0, twisted-1.14.3, requests-mock-1.12.1, respx-0.22.0
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 1 item

test_transcribe_audio.py::test_transcribe_audio_success 2025-08-07 16:07:05,966 - INFO - Audio file saved temporarily at: C:\Users\Karthik\AppData\Local\Temp\tmpkfrxnphb.mp3
2025-08-07 16:07:05,966 - INFO - Sending audio to Groq Whisper model...
2025-08-07 16:07:05,967 - INFO - Transcription successful.
2025-08-07 16:07:05,967 - INFO - Temporary file deleted: C:\Users\Karthik\AppData\Local\Temp\tmpkfrxnphb.mp3
2025-08-07 16:07:05,968 - INFO - HTTP Request: POST http://testserver/transcribe-audio "HTTP/1.1 200 OK"

[ API: /transcribe-audio (Success) ]
[ Inputs: Valid MP3 file ('test_audio.mp3') ]
[ Expected Output: Status 200, Body {'transcribed_text': 'This is a test transcription.'} ]
[ Actual Output: Status 200, Body {'transcribed_text': 'This is a test transcription.'} ]
[ Result: Success ]
PASSED
```

Page being tested: <http://localhost:8086/transcribe-audio>

Inputs:

- Request Method: **POST**
- Form Data
 - Field: audio_file
 - Value: sample-1.mp3
 - Content-Type: audio/mpeg
- Headers

Content-Type: multipart/form-data

accept: application/json

Expected Output:

- HTTP Status Code: 200

- **JSON Response:**

```
{
  "transcribed_text": "and who could buy hair without a conscious of the room, though strange
decided too, and some certain of the house. He had practified, but he was so disappeared on
the ten"
}
```

Actual Output:

- **HTTP Status Code:** 200

- **JSON Response:**

```
{
  "transcribed_text": "and who could buy hair without a conscious of the room, though strange
decided too, and some certain of the house. He had practified, but he was so disappeared on
the ten"
}
```

Result: Success

7. Chat Reminder Creation

```
def test_chat_with_agent_sets_reminder():
    api_endpoint = "/chat/ (Sets Reminder)"
    user_input = {"text": "Remind me to submit the report tomorrow EOD"}
    inputs_desc = f"User asks to set a reminder: {user_input}"
    # This is what the 'add_reminder' tool would return
    tool_response_content = "Reminder 'submit the report' set successfully for tomorrow and time EOD"
    # Expected final response from our FastAPI endpoint
    expected_status_code = 200
    expected_json = {"response": f"Reminder operation successful: {tool_response_content}", "reminders_list": None}
    expected_output_desc = f"Status {expected_status_code}, Body: {expected_json}"
    actual_output_desc = "Test did not run to completion."
    result = "Fail"
    try:
        mock_final_state = {
            "messages": [
                ToolMessage(content=tool_response_content, tool_call_id="tool_123")
            ]
        }

        # Patch 'main.agent_app.ainvoke' because it's an async function
        with patch('main.agent_app.ainvoke') as mock_ainvoke:
            mock_ainvoke.return_value = mock_final_state
            # Act
            response = client.post("/chat/", json=user_input)
            # Capture Actual Output
            actual_status_code = response.status_code
            actual_json = response.json()
            actual_output_desc = f"Status {actual_status_code}, Body: {actual_json}"
            # Assert
            assert actual_status_code == expected_status_code
            assert actual_json == expected_json
            # Ensure the mocked agent was called once with the correct initial state
            mock_ainvoke.assert_called_once()
            result = "Success"

    finally:
        print_test_report(api_endpoint, inputs_desc, expected_output_desc, actual_output_desc, result)
        if result == "Fail":
            pytest.fail("Test case failed. See report above for details.")
```

```
test_chat.py::test_chat_with_agent_sets_reminder 2025-08-07 16:19:54,731 - INFO - HTTP Request: POST http://testserver/chat/ "HTTP/1.1 200 OK"

[
  API being tested: /chat/ (Sets Reminder),
  Inputs: User asks to set a reminder: {'text': 'Remind me to submit the report tomorrow EOD'},
  Expected output: Status 200, Body: {'response': 'Reminder operation successful: Reminder 'submit the report' set successfully for tomorrow and time EOD', 'reminders_list': None},
  Actual Output: Status 200, Body: {'response': 'Reminder operation successful: Reminder 'submit the report' set successfully for tomorrow and time EOD', 'reminders_list': None},
  Result: Success
]
PASSED
```



Page being tested: <http://localhost:8086/chat>

Inputs:

- Request Method: **POST**
- **JSON Body**

```
{
  "text": "Remind me to call mom tomorrow at 3 PM about her doctor's appointment."
}
```

- **Headers**

Content-Type: application/json

accept: application/json

Expected Output:

- **HTTP Status Code: 200**

- **JSON Response:**

```
{
  "response": "Reminder operation successful: Reminder 'call mom about her doctor's appointment' set successfully for tomorrow and time 3 PM.",
  "reminders_list": null
}
```

Actual Output:

- **HTTP Status Code: 200**
- **JSON Response:**

```
{
  "response": "Reminder operation successful: Reminder 'call mom about her doctor's appointment' set successfully for tomorrow and time 3 PM.",
  "reminders_list": null
}
```

- **Response Headers:**

content-length: 162

content-type: application/json

date: Thu, 07 Aug 2025 10:47:14 GMT

server: uvicorn

Result: Success



Thank You!